

DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University



Submitted by:

**Aditya Wadhawan
(2401010023)**

B.Tech CSE (Core) SecA

Semester 4th

Submitted to

Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

Batch – 2024-2028

EXPERIMENT 1

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

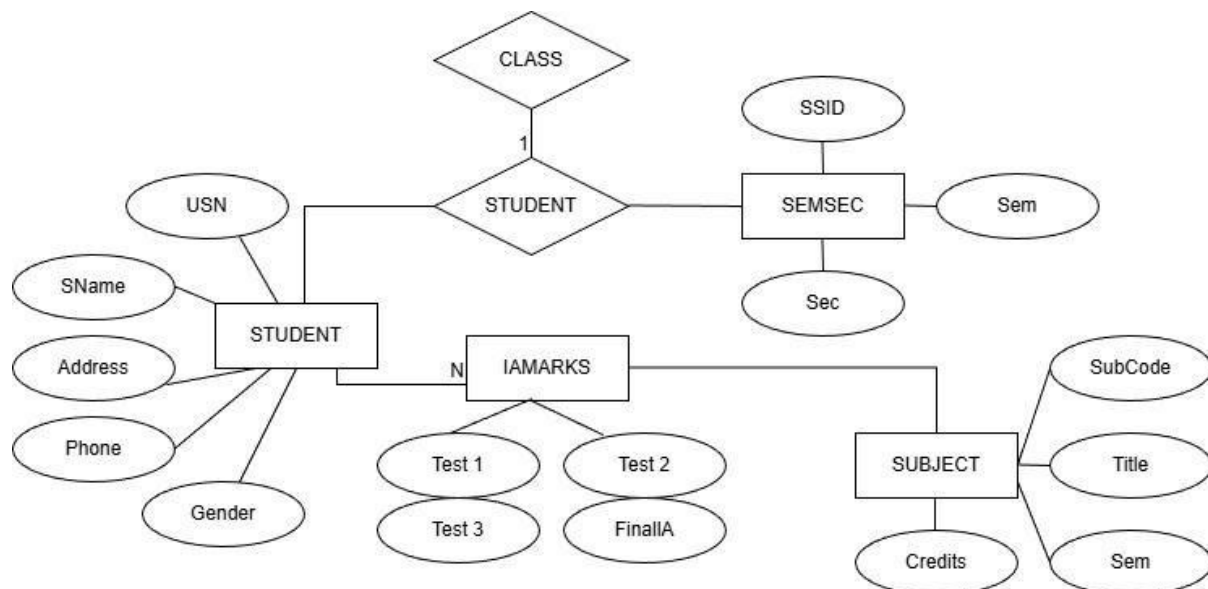
STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database

1. Introduction

The ER diagram is designed for the College Database system. It represents the structure of the database by identifying entities, their attributes, and the relationships between them. The main aim is to store and manage student details, semester-section information, subject details, and internal assessment marks.

2. Entities and Attributes

1. STUDENT

The STUDENT entity represents students of the college.

Attributes:

- **USN** (Primary Key)
- SName
- Address
- Phone
- Gender

USN uniquely identifies each student. No two students can have the same USN.

2. SEMSEC (Semester Section)

The SEMSEC entity represents a specific semester and section.

Attributes:

- **SSID** (Primary Key)
- Sem
- Sec

SSID uniquely identifies each semester-section combination.

3. SUBJECT

The SUBJECT entity represents subjects offered in different semesters.

Attributes:

- **Subcode** (Primary Key)
- Title
- Sem
- Credits

Subcode uniquely identifies each subject.

3. Relationships

1. CLASS Relationship

The CLASS relationship connects STUDENT and SEMSEC.

Meaning:

- Each student belongs to one semester and one section.

- One semester-section can have many students.

Cardinality:

- One SEMSEC → Many STUDENTS (1:M)

This relationship helps to identify which student belongs to which semester and section.

The relation table for CLASS contains:

- USN (Foreign Key)
- SSID (Foreign Key)

2. IAMARKS Relationship

The IAMARKS relationship connects STUDENT, SUBJECT, and SEMSEC.

Attributes:

- Test1
- Test2
- Test3
- FinallA Meaning:
- A student studies many subjects.
- A subject is studied by many students.
- Marks are recorded for each student in each subject.

Cardinality:

- Many-to-Many (M:N) between STUDENT and SUBJECT.

IAMARKS is considered a relationship with attributes because marks depend on both student and subject.

The relation table IAMARKS includes:

- USN (Foreign Key)
- Subcode (Foreign Key)
- SSID (Foreign Key)
- Test1
- Test2
- Test3
- FinallA

4. How the ER Diagram is Constructed

1. The main entities (STUDENT, SEMSEC, SUBJECT) are identified from the given data.
2. Primary keys are selected to uniquely identify each entity.
3. Relationships are identified:
 - o CLASS to connect student and semester-section.
 - o IAMARKS to store internal assessment marks.
4. Cardinality is defined:
 - o 1:M between SEMSEC and STUDENT.
 - o M:N between STUDENT and SUBJECT.
5. Attributes are connected to their respective entities and relationships.

5. Conclusion

The ER diagram clearly represents the structure of the college database system. It organizes student details, semester-section details, subject details, and internal assessment marks in a structured manner. The relationships ensure proper linkage between entities and help maintain data consistency.

EXPERIMENT 2

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:

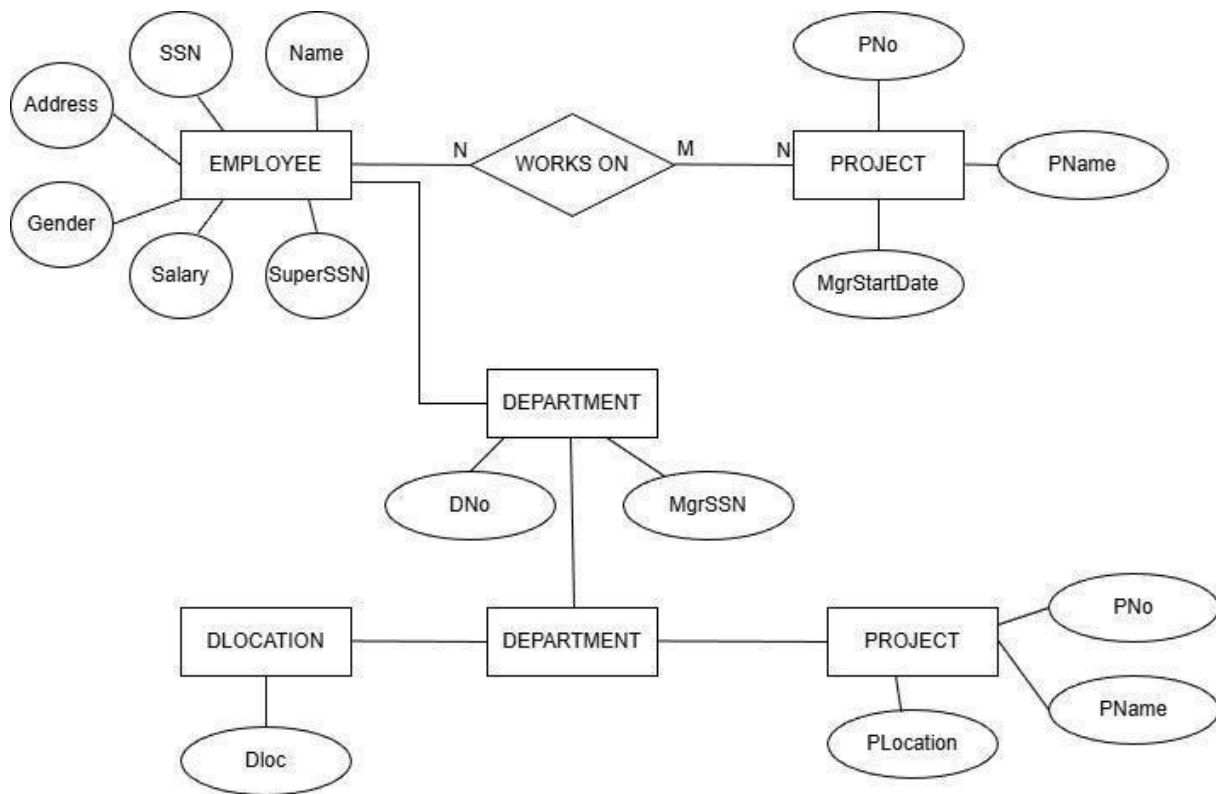
EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

DLOCATION (DNo,DLoc)

PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)



Explanation of ER Diagram – Company Database

1. Introduction

The ER diagram represents the structure of a Company Database system. It identifies the main entities such as employees, departments, projects, and locations, along with the relationships between them. The purpose of this ER diagram is to manage employee details, department information, project assignments, and work hours.

2. Entities and Attributes

1. EMPLOYEE

The EMPLOYEE entity represents the employees working in the company.

Attributes:

- **SSN** (Primary Key)
- Name
- Address
- Sex
- Salary
- SuperSSN
- DNo

SSN uniquely identifies each employee.

SuperSSN represents the supervisor of the employee (self-referencing relationship).

DNo indicates the department to which the employee belongs.

2. DEPARTMENT

The DEPARTMENT entity represents different departments in the company.

Attributes:

- **DNo** (Primary Key)
- DName
- MgrSSN
- MgrStartDate

DNo uniquely identifies each department.

MgrSSN represents the manager of the department.

3. PROJECT

The PROJECT entity represents projects handled by the company.

Attributes:

- **PNo** (Primary Key)
- PName
- PLocation

- DNo

PNo uniquely identifies each project.

DNo indicates which department controls the project.

4. DLOCATION

The DLOCATION entity represents the locations of departments.

Attributes:

- DNo
- DLoc

A department can have multiple locations.

3. Relationships

1. WORKS_ON Relationship

The WORKS_ON relationship connects EMPLOYEE and PROJECT.

Attribute:

- Hours Meaning:
- An employee can work on multiple projects.
- A project can have multiple employees working on it.

Cardinality:

- Many-to-Many (M:N)

The WORKS_ON table contains:

- SSN (Foreign Key)
 - PNo (Foreign Key)
 - Hours
-

2. Employee–Department Relationship

Each employee works in one department.

- One department can have many employees.

Cardinality:

- 1:M (One department → Many employees)
-

3. Department–Project Relationship

- One department controls many projects.
- Each project is controlled by one

department. Cardinality:

- 1:M (Department to Project)
-

4. Department–Location Relationship

- One department can have multiple locations.
 - Each location belongs to one department. Cardinality:
 - 1:M (Department to DLOCATION)
-

4. How the ER Diagram is Constructed

1. The main entities (EMPLOYEE, DEPARTMENT, PROJECT, DLOCATION) are identified.
2. Primary keys are assigned to uniquely identify each entity.
3. Relationships are identified:
 - WORKS_ON between EMPLOYEE and PROJECT.
 - Department–Employee relationship.
 - Department–Project relationship.
 - Department–Location relationship.
4. Cardinality is defined:
 - 1:M between Department and Employee.
 - 1:M between Department and Project.
 - 1:M between Department and Location.
 - M:N between Employee and Project.
5. Relationship attributes such as Hours are added to WORKS_ON.

5. Conclusion

The ER diagram clearly represents the structure of the company database. It shows how employees are assigned to departments, how departments manage projects, and how employees work on multiple projects. The relationships ensure proper organization and efficient management of company data.

Experiment 3

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string) **Item**(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;  
USE CustomerSaleDB;
```

```
CREATE TABLE Customer (  
    cust_id  
    INT PRIMARY KEY,  
    cust_name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Item (  
    item_id INT  
    PRIMARY KEY,    item_name  
    VARCHAR(100) NOT NULL,    price INT  
    NOT NULL CHECK (price > 0)  
);
```

```
CREATE TABLE Sale (  
    bill_no INT,    bill_date  
    DATE NOT NULL,    cust_id INT,  
    item_id INT,    qty_sold INT NOT NULL  
    CHECK (qty_sold > 0),  
    PRIMARY KEY (bill_no, item_id),  
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),  
    FOREIGN KEY (item_id) REFERENCES Item(item_id)  
);
```

	cust_id	cust_name
*	NULL	NULL

	item_id	item_name	price
*	NULL	NULL	NULL

	bill_no	bill_date	cust_id	item_id	qty_sold
*	NULL	NULL	NULL	NULL	NULL

2. Insert around 10 records in each of the tables.

	bill_no	cust_name	item_id
▶	1001	Ishaan	101
	1002	Mahin	102
	1003	Aditya	104
	1004	Ayush	105
	1005	Priya	108
	1006	Karan	110
	1007	Neha	103
	1008	Rohit	109
	1009	Simran	106
	1010	Vikas	107

INSERT INTO Customer VALUES

(1,'Ishaan'),
 (2,'Mahin'),
 (3,'Aditya'), (4,'Ayush'),
 (5,'Priya'),
 (6,'Karan'),
 (7,'Neha'),
 (8,'Rohit'),
 (9,'Simran'),
 (10,'Vikas');

INSERT INTO Item VALUES

(101,'Keyboard',50000),
 (102,'Mouse',500),
 (103,'Keyboard',800),
 (104,'Monitor',12000),
 (105,'USB Cable',200),
 (106,'Headphones',1500),
 (107,'Charger',900),
 (108,'Tablet',25000),
 (109,'Smartphone',30000),
 (110,'Pen Drive',700);

INSERT INTO Sale VALUES

(1001,CURDATE(),1,101,1),
 (1002,CURDATE(),2,102,2),
 (1003,CURDATE(),3,104,1),
 (1004,CURDATE(),4,105,3),
 (1005,CURDATE(),5,108,1),
 (1006,CURDATE(),6,110,4),
 (1007,CURDATE(),7,103,2),
 (1008,CURDATE(),8,109,1),
 (1009,CURDATE(),9,106,1), (1010,CURDATE(),10,107,2);

	cust_id	cust_name		item_id	item_name	price
▶	1	Ishaan	▶	101	Keyboard	50000
	2	Mahin		102	Mouse	500
	3	Aditya		103	Keyboard	800
	4	Ayush		104	Monitor	12000
	5	Priya		105	USB Cable	200
	6	Karan		106	Headphones	1500
	7	Neha		107	Charger	900 900
	8	Rohit		108	Tablet	25000
	9	Simran		109	Smartphone	30000
	10	Vikas		110	Pen Drive	700
★	NULL	NULL	★	NULL	NULL	NULL

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

	bill_no	bill_date	cust_id	item_id	qty_sold
▶	1001	2026-02-28	1	101	1
	1002	2026-02-28	2	102	2
	1003	2026-02-28	3	104	1
	1004	2026-02-28	4	105	3
	1005	2026-02-28	5	108	1
	1006	2026-02-28	6	110	4
	1007	2026-02-28	7	103	2
	1008	2026-02-28	8	109	1
	1009	2026-02-28	9	106	1
	1010	2026-02-28	10	107	2
★	NULL	NULL	NULL	NULL	NULL

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
    s.bill_no,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

	bill_no	qty_sold	price	final_amount
▶	1001	1	50000	50000
	1002	2	500	1000
	1003	1	12000	12000
	1004	3	200	600
	1005	1	25000	25000
	1006	4	700	2800
	1007	2	800	1600
	1008	1	30000	30000
	1009	1	1500	1500
	1010	2	900	1800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

	cust_id	cust_name
▶	1	Ishaan
	2	Mahin
	7	Neha
	3	Aditya
	9	Simran
	10	Vikas
	5	Priya
	8	Rohit
	6	Karan

6. Give a count of how many products have been bought by each customer

```
SELECT
    c.cust_id,
    c.cust_name,
    SUM(s.qty_sold) AS total_products_bought
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

	cust_id	cust_name	total_products_bought
▶	1	Ishaan	1
	2	Mahin	2
	3	Aditya	1
	4	Ayush	3
	5	Priya	1
	6	Karan	4
	7	Neha	2
	8	Rohit	1
	9	Simran	1
	10	Vikas	2

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

	item_id	item_name
▶	108	Tablet

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

	item_id	item_name	price
▶	101	Keyboard	50000
	102	Mouse	500
	104	Monitor	12000
	105	USB Cable	200
	108	Tablet	25000
	110	Pen Drive	700
	103	Keyboard	800
	109	Smartphone	30000
	106	Headphones	1500
	107	Charger	900

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE OR REPLACE VIEW Bill_View AS
SELECT
```

```

s.bill_no,
s.bill_date,
s.cust_id,
s.item_id,
i.price,
s.qty_sold,
(s.qty_sold * i.price) AS amount FROM
Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_View;

```

	bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
▶	1001	2026-02-28	1	101	50000	1	50000
	1002	2026-02-28	2	102	500	2	1000
	1003	2026-02-28	3	104	12000	1	12000
	1004	2026-02-28	4	105	200	3	600
	1005	2026-02-28	5	108	25000	1	25000
	1006	2026-02-28	6	110	700	4	2800
	1007	2026-02-28	7	103	800	2	1600
	1008	2026-02-28	8	109	30000	1	30000
	1009	2026-02-28	9	106	1500	1	1500
	1010	2026-02-28	10	107	900	2	1800

10. Create a view which lists the daily sales date wise for the last one week

```

CREATE OR REPLACE VIEW Weekly_Sales AS
SELECT
s.bill_date,
SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;

```

Bill_date:	2026-02-28
Total_sales:	126300

EXPERIMENT 4

Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,'Rahul');
- COMMIT;
- UPDATE class SET name = 'Abhijeet' WHERE id= '5';
- SAVEPOINT A;
- INSERT INTO class VALUES (6, 'Chris');
- SAVEPOINT B;
- INSERT INTO class VALUES (7, 'Bravo');
- SAVEPOINT C
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

CODE :

```
SET autocommit = 0;
```

```
CREATE TABLE class (  
    id INT,  
    name VARCHAR(50)  
);
```



```
INSERT INTO class VALUES (1,'Bravo');  
INSERT INTO class VALUES (2,'Alex');  
INSERT INTO class VALUES (4,'Cheng');
```

```
INSERT INTO class VALUES (5,'Rahul');  
COMMIT;
```

```
UPDATE class SET name='Abhijeet' WHERE id='5';
```

```
SAVEPOINT A;
```

```
INSERT INTO class VALUES (6,'Chris');
```

```
SAVEPOINT B;
```

```
INSERT INTO class VALUES (7,'Bravo');
```

```
SAVEPOINT C;
```

```
SELECT * FROM class;
```

```
ROLLBACK TO B;
```

```
SELECT * FROM class;
```

```
ROLLBACK TO A;
```

Table :

Result Grid



	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris
●	NULL	NULL

EXPERIMENT (4.2)

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

CODE :

```
SET autocommit = 0;
```

```
DROP TABLE IF EXISTS ACCOUNT_TXN;
```

```
CREATE TABLE ACCOUNT_TXN (  
  Acc_ID INT PRIMARY KEY,  
  Holder_Name VARCHAR(50),  
  Balance INT  
);
```

```
INSERT INTO ACCOUNT_TXN VALUES  
(101,'Aman',5000),  
(102,'Deepak',7000),  
(103,'Chirag',6000);
```

```
SELECT * FROM ACCOUNT_TXN;
```

```
START TRANSACTION;
```

```
INSERT INTO ACCOUNT_TXN VALUES (104,'Diya',4000);
```

```
SAVEPOINT S1;
```

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

INSERT INTO ACCOUNT_TXN VALUES (105,'Eshan',3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;

SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106,'Farah',4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500
WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

COMMIT;

OUTPUT :

Result Grid



Filter Rows:

	Acc_ID	Holder_Name	Balance
▶	101	Aman	6500
	102	Deepak	6000
	103	Chirag	6000
	104	Diya	4000
	106	Farah	4500
•	NULL	NULL	NULL

Experiment 5

```
EMPLOYEES (Employee_Id, First_Name, Last_Name, Email,  
Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id,  
Department_Id) CREATE database emp;
```

```
use emp;
```

```
CREATE TABLE EMPLOYEES (  
Employee_Id INT PRIMARY KEY,  
First_Name VARCHAR(50),  
Last_Name VARCHAR(50),  
Email VARCHAR(100),  
Phone_Number VARCHAR(20),  
Hire_Date DATE,  
Job_Id VARCHAR(20),  
Salary INT,  
Commission_Act DECIMAL(5,2),  
Manager_Id INT,  
Department_Id INT  
);
```

```
INSERT INTO EMPLOYEES VALUES
```

```
(101, 'John', 'Smith', 'john.smith@email.com', '9876543210', '2020-01-15', 'IT_PROG',  
6000, NULL, 100, 60),
```

```
(102, 'Sara', 'Connor', 'sara.connor@email.com', '9876543211', '2019-03-10', 'HR_REP',  
4500, NULL, 101, 70),
```

```
(103, 'Mike', 'Austin', 'mike.austin@email.com', '9876543212', '2021-06-20', 'SA_REP',  
5000, 0.10, 100, 80),
```

```
(104, 'Emma', 'Watson', 'emma.watson@email.com', '9876543213', '2018-11-05',  
'IT_PROG', 7000, NULL, 101, 60),
```

```
(105, 'Liam', 'Brown', 'liam.brown@email.com', '9876543214', '2022-02-18', 'SA_REP',  
4800, 0.15, 102, 80),
```

(106, 'Olivia', 'Davis', 'olivia.davis@email.com', '9876543215', '2020-07-25', 'HR_REP', 4200, NULL, 101, 70),

(107, 'Noah', 'Wilson', 'noah.wilson@email.com', '9876543216', '2017-09-30', 'IT_PROG', 8000, NULL, 100, 60),

(108, 'Ava', 'Taylor', 'ava.taylor@email.com', '9876543217', '2019-12-11', 'SA_REP', 4700, 0.12, 102, 80),

(109, 'Ethan', 'Anderson', 'ethan.anderson@email.com', '9876543218', '2021-04-14', 'IT_PROG', 5200, NULL, 101, 60),

(110, 'Sophia', 'Thomas', 'sophia.thomas@email.com', '9876543219', '2018-08-19', 'HR_REP', 4600, NULL, 100, 70),

(111, 'James', 'Jackson', 'james.jackson@email.com', '9876543220', '2020-05-23', 'SA_REP', 4900, 0.20, 102, 80),

(112, 'Isabella', 'White', 'isabella.white@email.com', '9876543221', '2019-01-30', 'IT_PROG', 6100, NULL, 101, 60),

(113, 'Lucas', 'Harris', 'lucas.harris@email.com', '9876543222', '2022-03-12', 'HR_REP', 4300, NULL, 100, 70),

(114, 'Mia', 'Martin', 'mia.martin@email.com', '9876543223', '2021-10-07', 'SA_REP', 5100, 0.18, 102, 80),

(115, 'William', 'Austin', 'william.austin@email.com', '9876543224', '2020-12-01', 'IT_PROG', 7500, NULL, 100, 60);

(a) Find out the employee id, names, salaries of all the employees

```
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

	Employee_Id	First_Name	Last_Name	Salary
▶	101	John	Smith	6000
	102	Sara	Connor	4500
	103	Mike	Austin	5000
	104	Emma	Watson	7000
	105	Liam	Brown	4800
	106	Olivia	Davis	4200
	107	Noah	Wilson	8000
	108	Ava	Taylor	4700
	109	Ethan	Anderson	5200
	110	Sophia	Thomas	4600
	111	James	Jackson	4900
	112	Isabella	White	6100
	113	Lucas	Harris	4300
	114	Mia	Martin	5100
	115	William	Austin	7500
★	NULL	NULL	NULL	NULL

EMPLOYEES 1

×

EMPLOYEES 2

EMPLOYEES 3

EMPLOYEES 4

EMPLOYEES 5

EMPL

(b) List out the employees who works under manager 100

SELECT *

FROM EMPLOYEES

WHERE Manager_Id = 100;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
▶	101	John	Smith	john.smith@email.com	9876543210	2020-01-15	IT_PROG	6000	NULL	100	60
	103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
	107	Noah	Wilson	noah.wilson@email.com	9876543216	2017-09-30	IT_PROG	8000	NULL	100	60
	110	Sophia	Thomas	sophia.thomas@email.com	9876543219	2018-08-19	HR_REP	4600	NULL	100	70
	113	Lucas	Harris	lucas.harris@email.com	9876543222	2022-03-12	HR_REP	4300	NULL	100	70
	115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1

EMPLOYEES 2 x

EMPLOYEES 3

EMPLOYEES 4

EMPLOYEES 5

EMPLOYEES 6

Apply

(c) Find the names of the employees who have a salary greater than or equal to 4800

SELECT First_Name, Last_Name

FROM EMPLOYEES

WHERE Salary >= 4800;

Result Grid		
Filter Rows:		
Export: Wrap C		
	First_Name	Last_Name
▶	John	Smith
	Mike	Austin
	Emma	Watson
	Liam	Brown
	Noah	Wilson
	Ethan	Anderson
	James	Jackson
	Isabella	White
	Mia	Martin
	William	Austin

EMPLOYEES 1 EMPLOYEES 2 EMPLOYEES 3 x EMPLOYEES 4

(d) List out the employees whose last name is 'AUSTIN'

SELECT *

FROM EMPLOYEES

WHERE Last_Name = 'AUSTIN';

Result Grid											
Filter Rows:											
Edit: Export/Import: Wrap Cell Content:											
	Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_Date	Job_Id	Salary	Commission_Act	Manager_Id	Department_Id
▶	103	Mike	Austin	mike.austin@email.com	9876543212	2021-06-20	SA_REP	5000	0.10	100	80
	115	William	Austin	william.austin@email.com	9876543224	2020-12-01	IT_PROG	7500	NULL	100	60
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

EMPLOYEES 1 EMPLOYEES 2 EMPLOYEES 3 EMPLOYEES 4 x EMPLOYEES 5 EMPLOYEES 6

(e) Find the names of the employees who works in departments 60,70 and 80

SELECT First_Name, Last_Name

FROM EMPLOYEES

WHERE Department_Id IN (60, 70, 80);

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	First_Name	Last_Name			
▶	John	Smith			
	Sara	Connor			
	Mike	Austin			
	Emma	Watson			
	Liam	Brown			
	Olivia	Davis			
	Noah	Wilson			
	Ava	Taylor			
	Ethan	Anderson			
	Sophia	Thomas			
	James	Jackson			
	Isabella	White			
	Lucas	Harris			
	Mia	Martin			
	William	Austin			

EMPLOYEES 1	EMPLOYEES 2	EMPLOYEES 3	EMPLOYEES 4	EMPLOYEES 5 ×
-------------	-------------	-------------	-------------	---------------

(f) Display the unique Manager_Id

SELECT DISTINCT Manager_Id

FROM EMPLOYEES;

Result Grid		Filter Rows:
	Manager_Id	
▶	100	
	101	
	102	

Experiment 6

(Exercise on updating records in table) Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due); Create database Client_master; use Client_master;

```
CREATE TABLE Client_master (  
    ClientNO VARCHAR(10) PRIMARY KEY,  
    Name VARCHAR(50),  
    Address VARCHAR(100),  
    City VARCHAR(50),    State  
    VARCHAR(50),    bal_due  
    DECIMAL(10,2)  
);
```

(a) Insert five records

```
INSERT INTO Client_master VALUES  
( 'C106', 'Anjali Kapoor', 'Sector 18', 'Noida', 'Uttar Pradesh', 4800),  
( 'C107', 'Vikas Malhotra', 'Connaught Place', 'Delhi', 'Delhi', 8000),  
( 'C108', 'Sneha Reddy', 'Banjara Hills', 'Hyderabad', 'Telangana', 6200),  
( 'C109', 'Arjun Nair', 'Marine Drive', 'Mumbai', 'Maharashtra', 5300),  
( 'C110', 'Karan Patel', 'SG Highway', 'Ahmedabad', 'Gujarat', 4500);
```

Result Grid						
Filter Rows:						
	ClientNO	Name	Address	City	State	bal_due
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3 × Client_master 4 Client_master 5 Client12 6

(b) Find the names of clients whose bal_due > 5000 .

SELECT Name

FROM Client_master

WHERE bal_due > 5000;

Result Grid	
Filter Rows:	
	Name
▶	Vikas Malhotra
	Sneha Reddy
	Arjun Nair

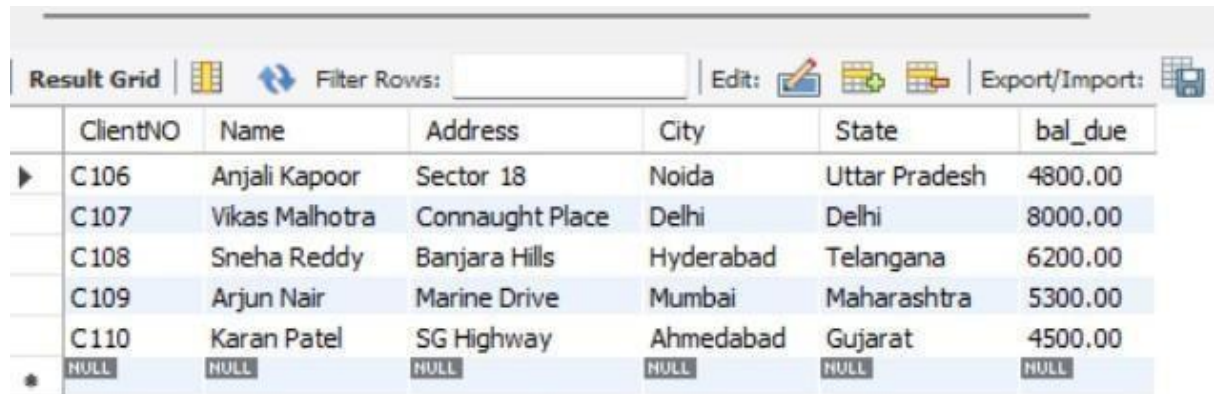
Client_master 3 Client_master 4 × Cl

(c) Change the bal_due of ClientNO “ C123” to Rs. 5100

UPDATE Client_master

SET bal_due = 5100

WHERE ClientNO = 'C123';



The screenshot shows a database application interface. At the top, there is a toolbar with icons for 'Result Grid', 'Filter Rows', 'Edit', and 'Export/Import'. Below the toolbar is a table with the following data:

	ClientNO	Name	Address	City	State	bal_due
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL

Client_master 3 Client_master 4 Client_master 5 × Client12 6

(d) Change the name of Client_master to Client12.

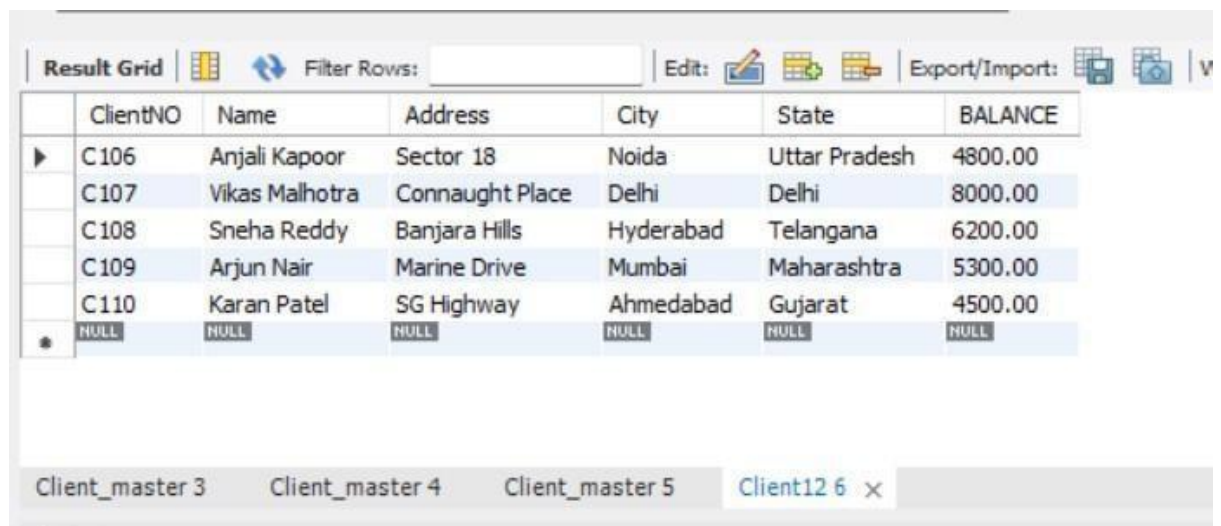
ALTER TABLE Client_master

RENAME TO Client12;

(e) Display the bal_due heading as “BALANCE”

SELECT ClientNO, Name, Address, City, State, bal_due AS BALANCE

FROM Client12;



The screenshot shows the same database application interface as before, but the column heading for the balance is now 'BALANCE'. The table data is identical to the previous screenshot:

	ClientNO	Name	Address	City	State	BALANCE
▶	C106	Anjali Kapoor	Sector 18	Noida	Uttar Pradesh	4800.00
	C107	Vikas Malhotra	Connaught Place	Delhi	Delhi	8000.00
	C108	Sneha Reddy	Banjara Hills	Hyderabad	Telangana	6200.00
	C109	Arjun Nair	Marine Drive	Mumbai	Maharashtra	5300.00
	C110	Karan Patel	SG Highway	Ahmedabad	Gujarat	4500.00
✱	NULL	NULL	NULL	NULL	NULL	NULL

At the bottom, the tab bar shows 'Client_master 3', 'Client_master 4', 'Client_master 5', and 'Client12 6 ×'.

Experiment 7

Objective: Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

- Insert five records.
- Give Increment of 25% salary for Mathematics Department.
- Perform Rollback command.
- Give increment of 15% salary for Commerce Department.
- Perform Commit command.

```
CREATE DATABASE IF NOT EXISTS school_management;
USE school_management;
SET SQL_SAFE_UPDATES = 0;
```

```
DROP TABLE IF EXISTS Teacher;
CREATE TABLE Teacher (
    TeacherName VARCHAR(50),
    DeptNo INT,
    DateOfJoining DATE,
    DeptName VARCHAR(50),
    LocationName VARCHAR(50),
    SalaryAmt INT
);
```

(a) Insert five records.

```
INSERT INTO Teacher (TeacherName, DeptNo, DateOfJoining, DeptName, LocationName,
    SalaryAmt) VALUES
('amit kumar', 10, '2021-01-10', 'mathematics', 'delhi', 40000),
('sunita devi', 20, '2020-03-15', 'commerce', 'mumbai', 35000),
('rahul sharma', 10, '2022-05-22', 'mathematics', 'delhi', 42000),
('priya nair', 30, '2019-11-01', 'science', 'kochi', 50000),
('aniket das', 20, '2023-02-14', 'commerce', 'kolkata', 32000);
```

(b) Give Increment of 25% salary for Mathematics Department.

```
START TRANSACTION;
UPDATE Teacher
SET SalaryAmt = SalaryAmt * 1.25
WHERE DeptName = 'mathematics';
SELECT * FROM Teacher WHERE DeptName = 'mathematics';
```

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	50000
	rahul sharma	10	2022-05-22	mathematics	delhi	52500

(c) Perform Rollback command.

```
ROLLBACK;
```

SELECT * FROM Teacher WHERE DeptName = 'mathematics';

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	40000
	rahul sharma	10	2022-05-22	mathematics	delhi	42000

(d) Give increment of 15% salary for Commerce Department.

```
START TRANSACTION;  
UPDATE Teacher  
SET SalaryAmt = SalaryAmt * 1.15  
WHERE DeptName = 'commerce';
```

(e) Perform Commit command.

```
COMMIT;  
SELECT * FROM Teacher;  
SETSQL_SAFE_UPDATES=1;
```

	TeacherName	DeptNo	DateOfJoining	DeptName	LocationName	SalaryAmt
▶	amit kumar	10	2021-01-10	mathematics	delhi	40000
	sunita devi	20	2020-03-15	commerce	mumbai	40250
	rahul sharma	10	2022-05-22	mathematics	delhi	42000
	priya nair	30	2019-11-01	science	kochi	50000
	aniket das	20	2023-02-14	commerce	kolkata	36800

Experiment 8

Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB) a) Insert five records.

b) Calculate total salesamount in each branch.

c) Calculate average salesamount in each branch.

d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09.

e) Display the name and DOB of salesman in alphabetical order of the month.

```
CREATE DATABASE IF NOT EXISTS sales_db;
USE sales_db;
```

```
DROP TABLE IF EXISTS Sales;
CREATE TABLE Sales (
    SalesNo INT PRIMARY KEY,
    SalesName VARCHAR(50),
    BranchName VARCHAR(50),
    SalesAmount INT,
    DateOfBirth DATE
);
```

(a) Insert five records.

```
INSERT INTO Sales (SalesNo, SalesName, BranchName, SalesAmount, DateOfBirth) VALUES
(101, 'arnav gupta', 'north', 45000, '1995-12-21'),
(102, 'sana sheikh', 'south', 32000, '1998-05-15'),
(103, 'kabir malhotra', 'north', 55000, '1996-12-09'),
(104, 'isha patel', 'east', 40000, '1997-12-05'),
(105, 'vihaan nair', 'south', 28000, '1999-08-22');
```

(b) Calculate total salesamount in each branch.

```
SELECT BranchName, SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY BranchName;
```

	BranchName	TotalSales
▶	north	100000
	south	60000
	east	40000

(c) Calculate average salesamount in each branch.

AIM:

```
SELECT BranchName, AVG(SalesAmount) AS AverageSales
FROM Sales
GROUP BY BranchName;
```

	BranchName	AverageSales
▶	north	50000.0000
	south	30000.0000
	east	40000.0000

(d) Display all the salesmen, DOB who are born in the month of December as day in characterformat i.e. 21-Dec-09.

```
SELECT SalesName, DATE_FORMAT(DateOfBirth, '%d-%b-%y')
AS FormattedDOB FROM
Sales
WHERE MONTH(DateOfBirth)=12;
```

	SalesName	FormattedDOB
▶	arnav gupta	21-Dec-95
	kabir malhotra	09-Dec-96
	isha patel	05-Dec-97

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
SELECT SalesName, DateOfBirth
FROM Sales
ORDER BY MONTHNAME(DateOfBirth) ASC;
```

	SalesName	DateOfBirth
▶	vihaan nair	1999-08-22
	arnav gupta	1995-12-21
	kabir malhotra	1996-12-09
	isha patel	1997-12-05
	sana sheikh	1998-05-15

Experiment 9

Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas are as follows:

Departments (dept no , dept name , dept_location);

Employees (emp_id , emp_name , emp_salary,dept_no);

- a) Develop a query to grant all privileges of employees table into departments table.
- b) Develop a query to grant some privileges of employees table into departments table.
- c) Develop a query to revoke all privileges of employees table from departments table.
- d) Develop query to revoke some privileges of employees table from departments table.
- e) Write a query to implement the save point.

```
CREATE DATABASE IF NOT EXISTS company_records;  
USE company_records;
```

```
CREATE TABLE IF NOT EXISTS Departments (  
    DeptNo INT PRIMARY KEY,  
    DeptName VARCHAR(50),  
    DeptLocation VARCHAR(50)  
);
```

```
CREATE TABLE IF NOT EXISTS Employees (  
    EmpId INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    EmpSalary INT,  
    DeptNo INT,  
    FOREIGN KEY (DeptNo) REFERENCES Departments(DeptNo)  
);
```

```
CREATE USER IF NOT EXISTS 'dept_manager'@'localhost' IDENTIFIED BY 'secure_pass123';
```

(a) Develop a query to grant all privileges of employees table into departments table.

```
GRANT ALL PRIVILEGES ON company_records.Employees TO 'dept_manager'@'localhost';
```

(b) Develop a query to grant some privileges of employees table into departments table.

AIM:

```
GRANT SELECT, UPDATE ON company_records.Employees TO 'dept_manager'@'localhost';
```

(c) Develop a query to revoke all privileges of employees table from departments table.

```
REVOKE ALL PRIVILEGES ON company_records.Employees FROM 'dept_manager'@'localhost';
```

(d) Develop a query to revoke some privileges of employees table from departments table.

```
REVOKE UPDATE ON company_records.Employees FROM 'dept_manager'@'localhost';
```

(e) Write a query to implement the save point.

```
START TRANSACTION;
```

```
INSERT INTO Employees (EmpId, EmpName, EmpSalary, DeptNo)
```

```
VALUES (501, 'rajat verma', 45000, NULL);
```

```
SAVEPOINT entry_alpha;
```

```
UPDATE Employees SET EmpSalary = 48000 WHERE EmpId = 501;
```

```
COMMIT;
```

	user	host
▶	dept_manager	localhost
	mysql.infoschema	localhost
	mysql.session	localhost
	mysql.sys	localhost
	root	localhost
	user1	localhost

	Grants for dept_manager@localhost
▶	GRANT USAGE ON *.* TO `dept_manager` @`l... GRANT SELECT ON `company_records`.`emplo...

Experiment 10

Using the tables “DEPARTMENTS” and “EMPLOYEES” from Lab 9 perform the following queries:

- Display the employee details, departments that the departments are same in both the emp and dept.
- Display the employee name and Department name by implementing a left outer join.
- Display the employee name and Department name by implementing a right outer join.
- Display the details of those who draw the salary greater than the average salary.

```
CREATE DATABASE IF NOT EXISTS company_joins;
USE company_joins;
```

```
CREATE TABLE IF NOT EXISTS Departments (
    DeptNo INT PRIMARY KEY,
    DeptName VARCHAR(50),
    DeptLocation VARCHAR(50)
);
```

```
CREATE TABLE IF NOT EXISTS Employees (
    EmpId INT PRIMARY KEY,
    EmpName VARCHAR(50),
    EmpSalary INT,
    DeptNo INT
);
```

```
INSERT INTO Departments (DeptNo, DeptName, DeptLocation) VALUES
(10, 'mathematics', 'block a'),
(20, 'commerce', 'block b'),
(30, 'science', 'block c'),
(40, 'humanities', 'block d');
```

```
INSERT INTO Employees (EmpId, EmpName, EmpSalary, DeptNo) VALUES
(701, 'amit kumar', 45000, 10),
(702, 'sunita devi', 55000, 20),
(703, 'rahul sharma', 38000, 10),
(704, 'priya nair', 60000, 30),
(705, 'vicky singh', 42000, NULL);
```

(a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT E.*, D.DeptName, D.DeptLocation
FROM Employees E
```

AIM:

JOIN Departments D ON E.DeptNo=D.DeptNo;

	EmpId	EmpName	EmpSalary	DeptNo	DeptName	DeptLocation
▶	701	amit kumar	45000	10	mathematics	block a
	702	sunita devi	55000	20	commerce	block b
	703	rahul sharma	38000	10	mathematics	block a
	704	priya nair	60000	30	science	block c

(b) Display the employee name and Department name by implementing a left outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
LEFT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

	EmpName	DeptName
▶	amit kumar	mathematics
	sunita devi	commerce
	rahul sharma	mathematics
	priya nair	science
	vicky singh	NULL

(c) Display the employee name and Department name by implementing a right outer join.

```
SELECT E.EmpName, D.DeptName
FROM Employees E
RIGHT OUTER JOIN Departments D ON E.DeptNo = D.DeptNo;
```

	EmpName	DeptName
▶	rahul sharma	mathematics
	amit kumar	mathematics
	sunita devi	commerce
	priya nair	science
	NULL	humanities

(d) Display the details of those who draw the salary greater than the average salary.

```
SELECT * FROM Employees
WHERE EmpSalary > (SELECT AVG(EmpSalary) FROM Employees);
```

	EmpId	EmpName	EmpSalary	DeptNo
▶	702	sunita devi	55000	20
	704	priya nair	60000	30
★	NULL	NULL	NULL	NULL